Diagrama de clases - UrbanRoute

Autor: Julio Reyes

urbanroute

Main

+ main(String[]): void

«crea»

urbanroute.vista

«extends JFrame»
InterfazUsuarioUI

- grafo: GrafoTransporte
- gestorArchivo: GestorArchivoCSV
- analizador: AnalizadorRedUrbana
- dibujo: Graph
- visor: SwingViewer
- botones, combos y área Resultados (Swing)

+ InterfazUsuarioUI()
- cargarArchivo(): void
- guardarArchivos(): boolean
- agregarEstacion(): void
- eliminarEstacion(): void
- agregarConexion(): void
- eliminarConexion(): void
- mostrarRutaMasCorta(): void
- mostrarSubredes(): void
- mostrarHubs(): void
- mostrarAyuda(): void
- dibujarGrafo(): void
- cerrarPrograma(): void

«dibuja con»

«librería externa»
GraphStream

Graph, SingleGraph
Node, Edge
SwingViewer, ViewPanel

(solo para dibujar el grafo)

grafo

gestorArchivo

analizador

urbanroute.control

1

GestorArchivoCSV

+ ENCABEZADO_ESTACIONES: String
+ ENCABEZADO_CONEXIONES: String
- TIEMPO_MAXIMO: int = 999
- archivoEstaciones: File
- archivoConexiones: File
- errores: String

+ cargarPorDefecto(GrafoTransporte): boolean
+ cargarArchivo(File, GrafoTransporte): boolean
+ guardar(GrafoTransporte, File, File): boolean
+ getErrores(): String
+ getArchivoEstaciones(): File
+ getArchivoConexiones(): File
- lineaAVertice(String, int, GrafoTransporte): void
- lineaAArista(String, int, GrafoTransporte): void
- armarTextoEstaciones(Estacion[]): String
- armarTextoConexiones(Estacion[]): String

1

AnalizadorRedUrbana

- INFINITO: int = Integer.MAX_VALUE
- grafo: GrafoTransporte
- tiempoUltimaRuta: int
- gradoHubs: int

+ AnalizadorRedUrbana(GrafoTransporte)
+ rutaMasCorta(String, String): String[]
+ getTiempoUltimaRuta(): int
+ buscarSubredes(): String[][]
+ buscarHubs(): String[]
+ getGradoHubs(): int
- recorrerBFS(Estacion[], int, int, int[]): void
- armarRuta(Estacion[], int[], int): String[]
- buscarIndice(Estacion[], String): int

«carga y guarda»

«crea»

grafo «solo lee»

«usa»

urbanroute.modelo

1

GrafoTransporte

- estaciones: TablaHashEstaciones
- cambiosSinGuardar: boolean

+ agregarEstacion(Estacion): boolean
+ agregarConexion(String, String, int): boolean
+ eliminarEstacion(String): boolean
+ eliminarConexion(String, String): boolean
+ buscarEstacion(String): Estacion
+ obtenerEstaciones(): Estacion[]
+ obtenerIds(): String[]
+ obtenerVecinos(String): String[]
+ obtenerTiempo(String, String): int
+ getCantidadEstaciones(): int
+ getCantidadConexiones(): int
+ quitarConexiones(): void
+ vaciar(): void
+ hayCambiosSinGuardar(): boolean

«usa»

Estacion

- id: String
- nombre: String
- linea: String
- zona: String
- capacidad: int
- conexiones: ListaEnlazada<Arista>

+ Estacion(String, String, String, String, int)
+ agregarConexion(Arista): boolean
+ eliminarConexion(String): boolean
+ getGrado(): int
+ getIdVecino(int): String
+ getTiempoConexion(int): int
+ getters y setters

conexiones

0..*

Arista

- idDestino: String
- tiempo: int

+ Arista(String, int)
+ getIdDestino(): String
+ getTiempo(): int

0..*

guarda

estaciones

«crea»

conexiones

«usa en BFS»

urbanroute.estructuras

1

TablaHashEstaciones

- CAPACIDAD_INICIAL: int = 16
- FACTOR_CARGA_MAXIMO: double = 0.75
- tabla: ListaEnlazada<Estacion>[]
- cantidad: int

+ insertar(Estacion): boolean
+ buscar(String): Estacion
+ eliminar(String): boolean
+ listarTodas(): Estacion[]
+ getCantidad(): int
- funcionHash(String): int
- rehash(): void

tabla

16..*

1

ListaEnlazada<T>

- primero: Nodo<T>
- ultimo: Nodo<T>
- tamano: int

+ insertar(T): void
+ eliminar(T): boolean
+ buscar(T): int
+ obtener(int): T
+ tamano(): int
+ estaVacia(): boolean

primero, ultimo

Cola<T>

- frente: Nodo<T>
- fin: Nodo<T>

+ encolar(T): void
+ desencolar(): T
+ estaVacia(): boolean

frente, fin

0..*

Nodo<T>

- dato: T
- siguiente: Nodo<T>

+ getDato(): T
+ setDato(T): void
+ getSiguiente(): Nodo<T>
+ setSiguiente(Nodo<T>): void

siguiente

0..1

0..*

Simbología

Asociación: la clase guarda un atributo del otro tipo

Composición: la clase crea y contiene a la otra

Agregación: la clase guarda objetos que existen por sí solos

Dependencia: la clase usa a la otra en un método

\+ público | - privado | «...» estereotipo | 0..* cero o muchos | 1 exactamente uno